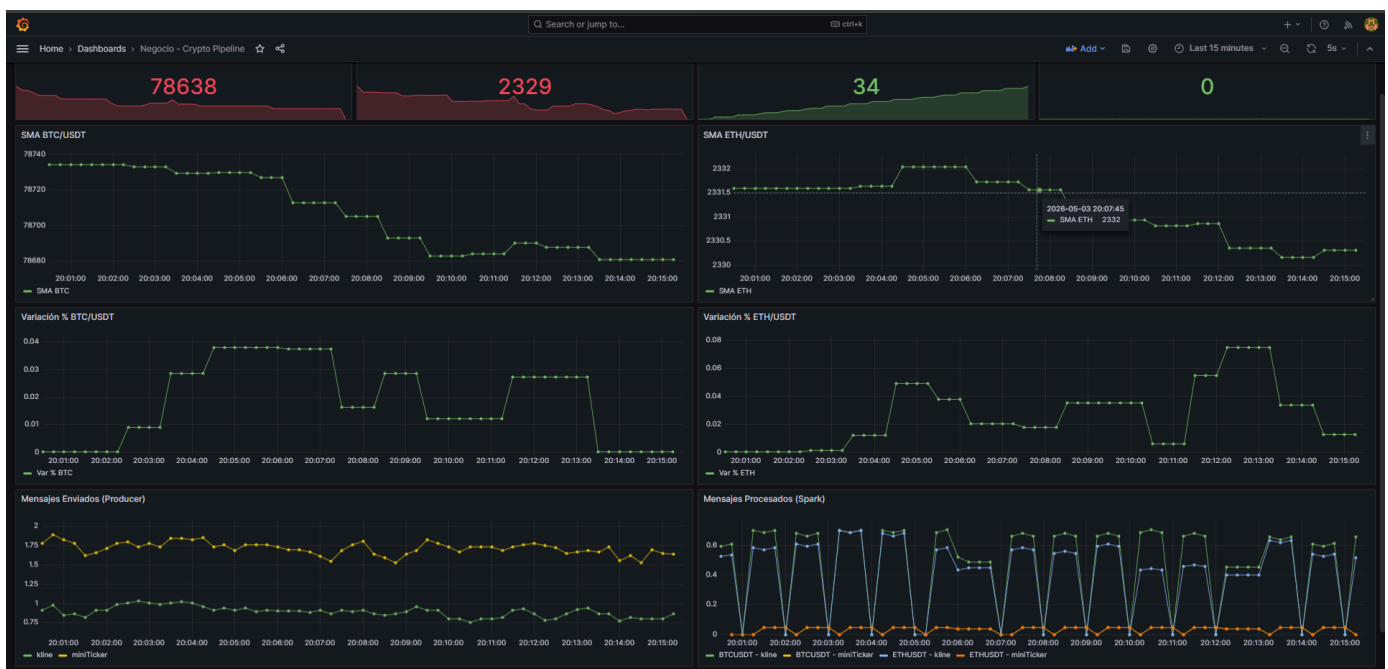


Crypto Exchange Pipeline



Memoria Técnica

Enlace a proyecto de GitHub :

https://github.com/FaduaHT/Crypto_Exchange.git

Fadoua Hathouti Lahrech

Big Data Aplicada

Índice

- 1. Resumen del Proyecto
- 2. Arquitectura Implementada
 - 2.1 Diagrama de Arquitectura
 - 2.2 Componentes y Dependencias
- 3. Decisiones de Diseño y Justificación
 - 3.1 Topics Kafka
 - 3.2 Particionado HDFS
 - 3.3 Ventanas Spark
 - 3.4 Métricas Prometheus y Alertas
 - 3.5 Esquemas Hive y Metastore
 - 3.6 Productor Python y Reconexión
- 4. Limitaciones del Sistema
 - 4.1 Limitaciones Arquitectónicas
 - 4.2 Limitaciones Funcionales
 - 4.3 Limitaciones Operativas
- 5. Posibles Mejoras y Líneas Futuras
 - 5.1 Mejoras de Alta Prioridad
 - 5.2 Mejoras de Media Prioridad
 - 5.3 Mejoras de Baja Prioridad

1. Resumen del Proyecto

Crypto Exchange Pipeline es una arquitectura de datos en tiempo real diseñada para ingestar, procesar, almacenar y visualizar información de mercado de criptomonedas. El sistema consume datos del WebSocket público de Binance, los canaliza a través de Apache Kafka, los procesa con Apache Spark Structured Streaming, los persiste en HDFS y expone tablas analíticas mediante Apache Hive. Toda la infraestructura es observable gracias a una stack de monitorización compuesta por Prometheus, Grafana, Node Exporter y cAdvisor, orquestada mediante Docker Compose.

1.1 Alcance Funcional

- **Ingesta en tiempo real** de precios spot (miniTicker) y velas OHLCV de 1 minuto (kline) para los pares BTC/USDT y ETH/USDT.
- **Procesamiento de ventanas deslizantes** para cálculo de métricas agregadas: media móvil (SMA), precio máximo/mínimo y variación porcentual.
- **Persistencia estructurada** en HDFS con formato Parquet y catálogo en Hive.
- **Observabilidad completa** con métricas de negocio (precios, throughput, volatilidad) e infraestructura (CPU, memoria, red por contenedor).

2. Arquitectura Implementada

2.1 Diagrama de Arquitectura

Arquitectura

Binance WS
(@miniTicker
@kline_1m)

Kafka
(Topics)

Spark Streaming
(Indicadores +
ventanas)

HDFS
(Parquet
partition)

Prometheus
+ Grafana
(Métricas
infra +
negocio)

- Fuentes: Binance WebSocket (@miniTicker, @kline_1m) — BTC/USDT + ETH/USDT
- Ingesta: Apache Kafka (Confluent 7.5)
- Procesamiento: Apache Spark 3.5 Structured Streaming
- Almacenamiento: HDFS 3.2 (Parquet, particionado por par/fecha)
- Observabilidad: Prometheus + Grafana + Node Exporter + cAdvisor
- Catálogo: Hive 4.0 (tablas externas sobre Parquet)

Diagrama de arquitectura implementada

2.2 Componentes y Dependencias

El sistema se despliega como **13 servicios Docker** sobre una red bridge personalizada (crypto-network):

☐	☒	proyectofinal_cryptoexchange	-	-	-	335.64%	2 hours ago			
☐	☒	spark-streaming	1bc875e63eb	proyectofinal_cryptoexchange-spark-streaming	8001:8001	16.56%	2 hours ago			
☐	☒	producer	aab21956ae16	proyectofinal_cryptoexchange-producer	8000:8000	0.68%	2 hours ago			
☐	☒	hive	90298dfdd61	apache/hive:4.0.0	10000:10000 Show all ports (2)	0.97%	2 hours ago			
☐	☒	grafana	b094c002dd21	grafana/grafana:10.3.1	3000:3000	0.04%	2 hours ago			
☐	☒	kafka	4ca987d70137	confluentinc/cp-kafka:7.5.0	9092:9092 Show all ports (2)	1.05%	2 hours ago			
☐	☒	spark-worker	83f825aa38a6	apache/spark:3.5.0	8082:8081	306.47%	2 hours ago			
☐	☒	datanode	96e09e101f22	bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8	9864:9864	0.34%	2 hours ago			
☐	☒	prometheus	dbd4addf56f0	prom/prometheus:v2.50.1	9090:9090	0.64%	2 hours ago			
☐	☒	cadvisor	3116d0fa9fe9	cadvisor/cadvisor:v0.47.2	8081:8080	0.6%	2 hours ago			
☐	☒	node-exporter	5b8e45709146	prom/node-exporter:v1.7.0	9100:9100	0%	2 hours ago			
☐	☒	portainer	fd427e815c5b	portainer/portainer-ce:2.19.5	9000:9000	0.03%	2 hours ago			
☐	☒	zookeeper	5ec654971ba8	confluentinc/cp-zookeeper:7.5.0	2181:2181	0.19%	2 hours ago			
☐	☒	spark-master	e3483dab8a8d	apache/spark:3.5.0	7077:7077 Show all ports (3)	6.27%	2 hours ago			
☐	☒	namenode	379529c980ef	bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8	9001:9000 Show all ports (2)	1.8%	2 hours ago			

3. Decisiones de Diseño y Justificación

3.1 Topics Kafka

Decision: Dos topics independientes (miniTicker y kline) en lugar de un único topic genérico.

Justificación técnica:

- **Separación de frecuencias:** mini Ticker genera ~1 mensaje/segundo por par, mientras que kline genera ~1 mensaje/minuto. Unificarlas en un solo topic provocaría desbalance de consumo y dificulta el tuning de Spark (batch sizes desiguales).
- **Semántica diferente:** Los mensajes miniTicker requieren agregación con ventanas temporales (SMA, volatilidad), mientras que kline se trata como ingestión raw historica. Spark Structured Streaming puede aplicar lógicas distintas a streams independientes de forma natural.
- **Key por par:** Se utiliza par (BTCUSDT / ETHUSDT) como clave de partición. Esto garantiza que todos los eventos de un mismo par caigan en la misma partición, preservando el orden relativo por activo y facilitando joins futuros.

Nota: Se habilita `auto.create.topics.enable=true` para facilitar el despliegue local. En un entorno productivo se definirán explícitamente con replicación mayor o igual a 2.

3.2 Particionado HDFS

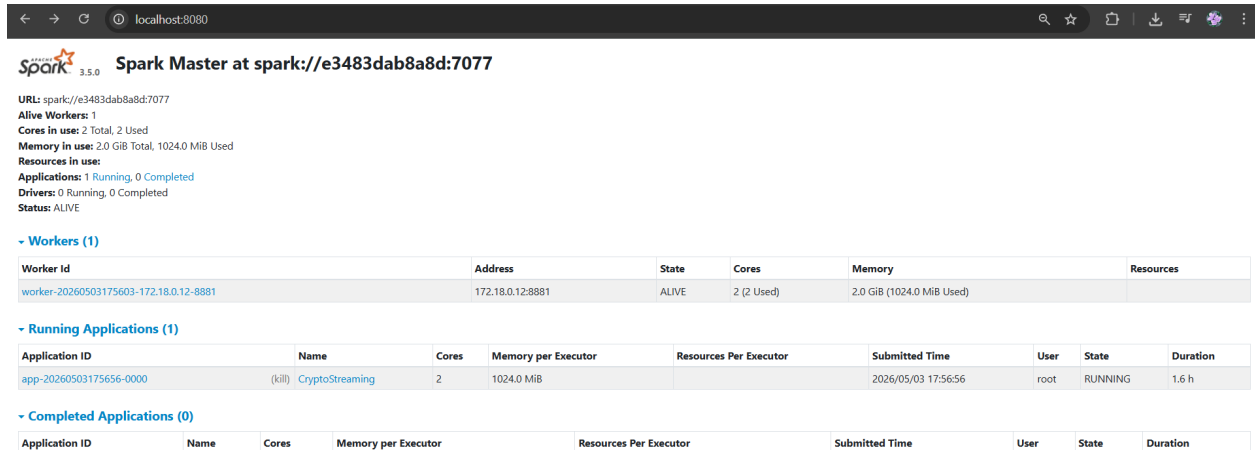
Decisión: El pipeline persiste en HDFS con formato Parquet en rutas fijas (/data/cripto/miniTicker y /data/cripto/kline). Las columnas par y fecha existen en el esquema, pero la escritura actual no utiliza particionado explícito por directorios.

Justificación técnica:

- **Formato Parquet:** Elegido por su eficiencia en compresión (snappy/gzip), proyección de columnas y compatibilidad nativa con Spark SQL y Hive. Reduce drásticamente el I/O respecto a formatos row-based como CSV o JSON.
- **Replicación factor 1:** Justificado por tratarse de un cluster local de un solo nodo físico. Aumentar la replicación no aportaría tolerancia a fallos de hardware (solo un datanode) e incrementa el uso de disco innecesariamente.
- **Permisos deshabilitados (dfs.permissions.enabled=false):** Simplifica la interoperabilidad entre contenedores Spark y Hive, evitando problemas de UserGroupInformation en entornos Docker donde los UIDs no coinciden.
- **Particionado por directorios:** Aunque el esquema de datos incluye par y fecha, la escritura Spark actual no aplica `.partitionBy()`. Esto es una decisión pragmática para la demo local, ya que con los volúmenes de prueba (~2 msg/s) el overhead de

crear múltiples directorios pequeños supera el beneficio de pruning. En producción se implementaría `partitionBy("par", "fecha")`.

3.3 Ventanas Spark



URL: spark://e3483dab8a8d:7077

Alive Workers: 1
Cores in use: 2 Total, 2 Used
Memory in use: 2.0 GiB Total, 1024.0 MiB Used
Resources in use:
Applications: 1 Running, 0 Completed
Drivers: 0 Running, 0 Completed
Status: ALIVE

Workers (1)

Worker Id	Address	State	Cores	Memory	Resources
worker-20260503175603-172.18.0.12-8881	172.18.0.12:8881	ALIVE	2 (2 Used)	2.0 GiB (1024.0 MiB Used)	

Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260503175656-0000	(kill) CryptoStreaming	2	1024.0 MiB		2026/05/03 17:56:56	root	RUNNING	1.6 h

Completed Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

Decisión: Para `miniTicker` se utiliza una **ventana deslizante (sliding window)** de **2 minutos con slide de 30 segundos**, con watermark de 1 minuto.

Justificación técnica:

- **Sliding window vs Tumbling window:** Se elige ventana deslizante porque permite obtener lecturas de agregaciones más frecuentes (cada 30s) que el tamaño de la ventana misma (2 min). Esto es crítico para un dashboard de trading donde el operador quiere ver actualizaciones frecuentes, no solo cada 2 minutos.
- **Watermark de 1 minuto:** Permite tolerar desorden en los eventos (out-of-order) hasta 60 segundos. Dado que el producer consume de un WebSocket remoto (Binance) y la red puede introducir latencia variable, el watermark evita que eventos ligeramente tardíos se descarten.
- **2 min / 30 seg en lugar de 5 min / 1 min:** Aunque la documentación de diseño original planteaba 5 minutos con slide de 1 minuto para producción, la implementación local usa 2 min / 30 seg. Esta reducción permite visualizar rápidamente el funcionamiento del pipeline durante la demo sin esperar ciclos largos. La arquitectura soporta cambiar estos parámetros sin modificar la lógica de negocio.
- **Kline sin ventana:** El stream de velas (`kline`) se procesa como ingestión raw porque cada mensaje ya es una agregación de 1 minuto realizada por Binance. Aplicar una ventana adicional sería redundante.

- **Trigger processingTime="60s"**: Escribe micro-batches cada minuto, equilibrando latencia con overhead de Spark. Un intervalo menor (ej. 10s) saturaría HDFS con archivos Parquet muy pequeños en este volumen.
- **Checkpoints en HDFS**: Se configuran en `hdfs://namenode:9000/data/cripto/checkpoints`. Esto garantiza tolerancia a fallos: si el contenedor `spark-streaming` se reinicia, Spark recupera el offset de Kafka y el estado de las ventanas desde el último checkpoint.

3.4 Métricas Prometheus y Alertas

Decisión: Se diseñan al menos 6 métricas de negocio personalizadas, además de métricas de infraestructura, con 3 alertas activas.

3.4.1 Métricas de Negocio (Pipeline)

Tabla 3 Metricas de negocio personalizadas

Metrica	Tipo	Fuente	Proposito
<code>crypto_precio_actual{par, tipo_stream}</code>	Gauge	Producer	Ultimo precio conocido por par. Fundamental para el dashboard.
<code>pipeline_mensajes_enviados_total{topic}</code>	Counter	Producer	Throughput de salida. Permite detectar caidas del producer.
<code>websocket_reconexiones_total</code>	Counter	Producer	Indicador de estabilidad de red. Picos indican problemas con Binance.
<code>crypto_precio_sma{par}</code>	Gauge	Spark	Media movil calculada por ventana. Metrica analitica clave.
<code>crypto_variacion_pct{par}</code>	Gauge	Spark	Volatilidad intra-ventana. Dispara alerta si supera umbrales.
<code>pipeline_mensajes_procesados_total{topic, par}</code>	Counter	Spark	Throughput de entrada a Spark. Permite detectar lag o caidas.

Métricas de negocio personalizadas

3.4.2 Métricas de Infraestructura

- **Node Exporter (:9100)**: CPU, RAM, disco, red del host Linux. Permite correlacionar problemas del pipeline con saturacion de recursos fisicos.

```
localhost:9100/metrics

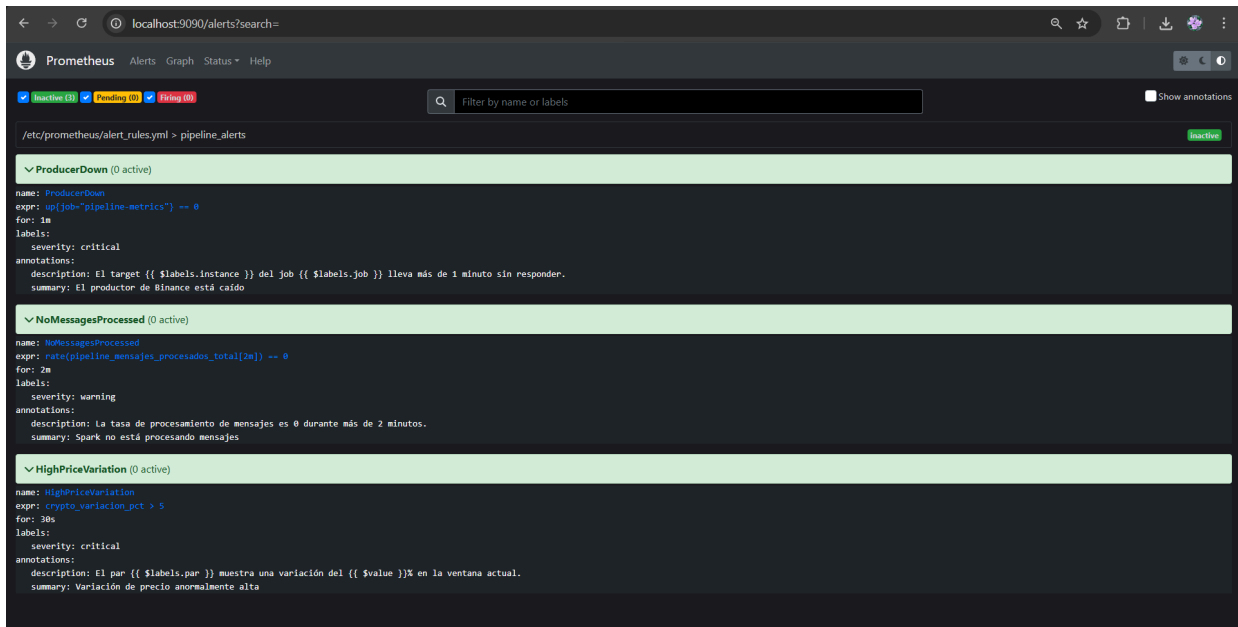
# HELP go_gc_duration_seconds A summary of the pause duration of garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 4.0651e-05
go_gc_duration_seconds{quantile="0.25"} 5.4399e-05
go_gc_duration_seconds{quantile="0.5"} 7.1749e-05
go_gc_duration_seconds{quantile="0.75"} 9.9969e-05
go_gc_duration_seconds{quantile="1"} 0.001100214
go_gc_duration_seconds_sum 0.041086219
go_gc_duration_seconds_count 487
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 9
# HELP go_info Information about the Go environment.
# TYPE go_info gauge
go_info(version="go1.21.4") 1
# HELP go_memstats_alloc_bytes Number of bytes allocated and still in use.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 2.268656e+06
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated, even if freed.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 8.97602712e+08
# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.589135e+06
# HELP go_memstats_frees_total Total number of frees.
# TYPE go_memstats_frees_total counter
go_memstats_frees_total 1.1820588e+07
# HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata.
# TYPE go_memstats_gc_sys_bytes gauge
go_memstats_gc_sys_bytes 4.405408e+06
# HELP go_memstats_heap_alloc_bytes Number of heap bytes allocated and still in use.
# TYPE go_memstats_heap_alloc_bytes gauge
go_memstats_heap_alloc_bytes 2.268656e+06
# HELP go_memstats_heap_idle_bytes Number of heap bytes waiting to be used.
# TYPE go_memstats_heap_idle_bytes gauge
go_memstats_heap_idle_bytes 3.203072e+06
# HELP go_memstats_heap_inuse_bytes Number of heap bytes that are in use.
# TYPE go_memstats_heap_inuse_bytes gauge
go_memstats_heap_inuse_bytes 4.562944e+06
# HELP go_memstats_heap_objects Number of allocated objects.
# TYPE go_memstats_heap_objects gauge
go_memstats_heap_objects 15897
# HELP go_memstats_heap_released_bytes Number of heap bytes released to OS.
# TYPE go_memstats_heap_released_bytes gauge
go_memstats_heap_released_bytes 2.318336e+06
# HELP go_memstats_heap_sys_bytes Number of heap bytes obtained from system.
# TYPE go_memstats_heap_sys_bytes gauge
go_memstats_heap_sys_bytes 7.76016e+06
# HELP go_memstats_last_gc_time_seconds Number of seconds since 1970 of last garbage collection.
# TYPE go_memstats_last_gc_time_seconds gauge
go_memstats_last_gc_time_seconds 1.7778374645435722e+09
# HELP go_memstats_lookups_total Total number of pointer lookups.
# TYPE go_memstats_lookups_total counter
go_memstats_lookups_total 0
# HELP go_memstats_mallocs_total Total number of mallocs.
# TYPE go_memstats_mallocs_total counter
go_memstats_mallocs_total 1.1836485e+07
# HELP go_memstats_mcache_inuse_bytes Number of bytes in use by mcache structures.
# TYPE go_memstats_mcache_inuse_bytes gauge
go_memstats_mcache_inuse_bytes 1200
# HELP go_memstats_mcache_sys_bytes Number of bytes used for mcache structures obtained from system.
# TYPE go_memstats_mcache_sys_bytes gauge
go_memstats_mcache_sys_bytes 15600
# HELP go_memstats_mspan_inuse_bytes Number of bytes in use by mspan structures.
# TYPE go_memstats_mspan_inuse_bytes gauge
go_memstats_mspan_inuse_bytes 82152
# HELP go_memstats_mspan_sys_bytes Number of bytes used for mspan structures obtained from system.
# TYPE go_memstats_mspan_sys_bytes gauge
go_memstats_mspan_sys_bytes 97776
# HELP go_memstats_next_gc_bytes Number of heap bytes when next garbage collection will take place.
```

- **cAdvisor (:8080):** Métricas por contenedor Docker (uso de CPU, memoria, red, filesystem). Esencial para debuggear que servicio está consumiendo recursos.



3.4.3 Alertas Configuradas

Prometheus Alerts			
Rules			
pipeline_alerts			
Rule	Interval: 15.0s	12.853s ago	0.480ms
<pre>alert: ProductDown expr: pipeline_metrics == 0 for: 1m labels: severity: critical annotations: description: El target {{ \$target }} del job {{ \$job }} lleva más de 1 minuto sin responder. summary: El productor de binance está caído.</pre>	OK	12.853s ago	0.285ms
<pre>alert: NoMessageReceived expr: rate(pipeline_messages_received[2m]) == 0 for: 2m labels: severity: warning annotations: description: La tasa de procesamiento de mensajes es 0 durante más de 2 minutos. summary: Spark no está procesando mensajes</pre>	OK	12.859s ago	0.112ms
<pre>alert: HighVariance expr: crypto_variance_pct > 5 for: 5m labels: severity: critical annotations: description: El par {{ \$pair }} muestra una variación del {{ \$variance }}% en la ventana actual. summary: Variación de precio anormalmente alta</pre>	OK	12.859s ago	0.053ms



Reglas de alerta configuradas

Scrape interval 15s: Balance entre granularidad y carga. En producción con alta cardinalidad podría aumentarse a 30s.

3.5 Esquemas Hive y Metastore

Decisión: Tablas externas (EXTERNAL TABLE) con metastore Derby embebido.

Justificación técnica:

- **Tablas externas:** Los datos residen en HDFS y son gestionados por Spark. Si Hive se destruye o reinicia, los datos y el esquema persistente en HDFS no se ven afectados. Esto desacopla el catálogo del procesamiento.
- **Derby embebido:** Justificado por simplicidad en despliegue local. No requiere un servicio PostgreSQL/MySQL adicional. En producción se migraría a un metastore remoto (PostgreSQL) para permitir alta disponibilidad y acceso concurrente.
- **Formato Parquet:** Las tablas apuntan directamente a archivos Parquet escritos por Spark, sin necesidad de transformaciones adicionales. Esto habilita consultas SQL interactivas sobre datos en tiempo real.

3.6 Productor Python y Reconexión

Decisión: Productor propio en Python 3.11 con reconexión automática y backoff exponencial.

Justificación técnica:

- **WebSocket nativo vs Kafka Connect:** Aunque Kafka Connect con un conector WebSocket existiría, un producer propio permite enriquecer los payloads (añadir timestamp local, normalizar nombres de campos) y exponer métricas de negocio personalizadas directamente en el mismo proceso.
- **Backoff exponencial (1s → 60s):** Evita saturar la API de Binance con reintentos agresivos cuando hay problemas de red. El tope de 60s es razonable para mantener la conexión sin ser bloqueado por rate-limits.
- **Serialización JSON:** Binance ya envió JSON, por lo que no hay overhead de serialización adicional. Se enriquece con metadatos antes de enviar a Kafka.

4. Limitaciones del Sistema

4.1 Limitaciones Arquitectonicas

1. **Single Point of Failure en Kafka:** Sólo un broker Kafka. Si cae, toda la ingesta se detiene. No hay particionado con replicación mayor o igual a 2.
2. **HDFS single-node:** Un solo NameNode y DataNode. Cualquier fallo del volumen Docker implica pérdida total de datos históricos (aunque los checkpoints permiten reiniciar desde el offset de Kafka).
3. **Metastore Derby embebido:** No soporta acceso concurrente desde múltiples clientes Hive. En escenarios multi-usuario se convertiría en cuello de botella.
4. **Spark en modo standalone:** No utiliza YARN ni Kubernetes. El scheduling es básico y no soporta colas de recursos ni autoscaling.
5. **Sin esquema de particionado HDFS:** Como se mencionó, la falta de `partitionBy("par", "fecha")` en la escritura Spark implica que las consultas Hive no pueden beneficiarse de partition pruning, escaneando archivos innecesarios.

4.2 Limitaciones Funcionales

1. **Sólo 2 pares de trading:** BTC/USDT y ETH/USDT. Ampliar a mas pares requeriría reconfigurar el producer y ajustar recursos de Spark.
2. **Solo 2 streams de Binance:** No se consumen depth (order book), trades individuales ni datos de futuros.
3. **Alertas estáticas:** El umbral de `HighPriceVariation` esta hardcodeado al 5%. No se adapta a la volatilidad histórica de cada par.
4. **Sin exactly-once semantics:** Spark Structured Streaming si soporta exactly-once con Kafka, pero la configuración actual no ha sido verificada explícitamente contra duplicados en escenarios de fallo.
5. **Sin mecanismo de backpressure visible:** Aunque Spark tiene backpressure interno, no hay métrica explícita de "Kafka consumer lag" en el dashboard, lo que dificulta detectar si Spark va por detrás del producer.

4.3 Limitaciones Operativas

1. **Credenciales en .env:** La contraseña de Grafana (admin/admin) está en un archivo plano. En producción debería integrarse con un secret manager (HashiCorp Vault, AWS Secrets Manager).
2. **Sin CI/CD:** No hay pipelines de testing ni despliegue automático.
3. **Sin data retention policies:** Kafka y HDFS crecería indefinidamente. No hay políticas de compactación ni TTL configuradas.

5. Posibles Mejoras y Líneas Futuras

5.1 Mejoras de Alta Prioridad (Corto Plazo)

1. **Implementar particionado HDFS:** Añadir `.partitionBy("par", "fecha")` en la escritura Spark para habilitar partition pruning y mejorar rendimiento de consultas históricas.
2. **Métrica de Kafka Lag:** Exponer `kafka_consumer_lag` para detectar cuando Spark va por detrás del producer. Se puede implementar con `kafka-consumer-groups.sh` o librerías como `kafka-lag-exporter`.
3. **Ampliar a más pares:** Parametrizar la lista de pares en una variable de entorno del producer para facilitar la configuración dinámica.
4. **Exactly-once verification:** Auditar la configuración de Spark + Kafka para garantizar exactly-once semánticos, especialmente en la escritura a HDFS.

5.2 Mejoras de Media Prioridad (Medio Plazo)

5. **Cluster Kafka multi-broker:** Desplegar 3 brokers con replicación factor 3 y ZooKeeper en ensemble para tolerancia a fallos.
6. **Metastore remoto:** Migrar de Derby embebido a PostgreSQL para soportar múltiples conexiones concurrentes a HiveServer2.
7. **Data retention y compresión:** Configurar políticas de retención en Kafka (ej. 7 días) y en HDFS (compresión Snappy + archivado a S3/GCS para datos antiguos).
8. **Autoscaling Spark:** Migrar el despliegue de Spark standalone a Kubernetes (Spark-on-K8s) para habilitar autoscaling de executors según carga.
9. **Alertas dinámicas:** Implementar umbrales adaptativos basados en desviación estándar histórica (Bollinger Bands) en lugar de un umbral fijo del 5%.

5.3 Mejoras de Baja Prioridad / Investigacion (Largo Plazo)

10. **ML en streaming:** Entrenar un modelo de detección de anomalías (Isolation Forest o LSTM) directamente sobre el stream de Spark para identificar movimientos sospechosos (pump and dump).
11. **Fuentes adicionales:** Integrar CoinGecko, CoinMarketCap o APIs de exchanges descentralizados (Uniswap) para enriquecimiento de datos y detección de arbitraje.
12. **Delta Lake / Apache Iceberg:** Migrar el almacenamiento de Parquet raw a Delta Lake para habilitar ACID transactions, time-travel y merge de streams con batch.

13. **API REST de consulta:** Exponer una API (FastAPI/Flask) sobre Hive/Spark SQL para que aplicaciones externas consuman los indicadores calculados.
14. **Cost monitoring:** En despliegue cloud, integrar métricas de coste por GB procesado/almacenado.